

# Smart Disaster Response MIS

Indexing Strategy & Query Performance Report

## DELIVERABLE — PERFORMANCE ANALYSIS

Course: Database Systems  
Campus: FAST-NUCES, Islamabad  
Semester: Spring 2026  
Document: Indexing & Performance

## Table of Contents

---

1. Indexing Strategy Rationale
  - 1.1 Key Index Definitions
2. Query Performance Benchmarks
  - 2.1 Filter Performance — Logical Reads
  - 2.2 Execution Time Impact — Latency
3. Write Overhead Analysis
4. Conclusion

# 1. Indexing Strategy Rationale

- **Primary Key Clustering:** All tables use IDENTITY-based clustered primary keys for efficient row retrieval.
- **Search Optimisation:** Single-column indexes on high-cardinality columns frequently appearing in WHERE clauses.
- **Composite Indexing:** Multi-column indexes for complex dashboard queries filtering on multiple attributes simultaneously.

## 1.1 Key Index Definitions

| Index Name                | Table               | Columns                            | Rationale                         |
|---------------------------|---------------------|------------------------------------|-----------------------------------|
| IX_ER_DisasterType_Status | EmergencyReports    | (disaster_type, status)            | Primary 'Active Incidents' filter |
| IX_ER_Location_Severity   | EmergencyReports    | (location, severity_level)         | Geographic incident analysis      |
| IX_FT_Type_Timestamp      | FinanceTransactions | (type, timestamp)                  | Monthly financial statements      |
| IX_AL_ActionTimestamp     | AuditLog            | (action_timestamp)                 | Date-range audit queries          |
| IX_AL_Table_Timestamp     | AuditLog            | (table_affected, action_timestamp) | Table-specific audit history      |
| IX_RA_Report_Status       | ResourceAllocations | (report_id, status)                | Allocation pipeline queries       |

Table 1 — Index definitions with rationale

## 2. Query Performance Benchmarks

Benchmarks run on ~1,000 records using SET STATISTICS IO ON and SET STATISTICS TIME ON on MS SQL Server 2022.

### 2.1 Filter Performance — Logical Reads

| Query Type                           | Before Indexing | After Indexing | Improvement |
|--------------------------------------|-----------------|----------------|-------------|
| Incidents by Type & Status           | 18 reads        | 3 reads        | 83%         |
| Incidents by Location & Severity     | 18 reads        | 4 reads        | 78%         |
| Audit Logs by Date Range             | 32 reads        | 6 reads        | 81%         |
| Audit Logs by Table & Date           | 32 reads        | 4 reads        | 88%         |
| Resource Allocations (Report+Status) | 8 reads         | 2 reads        | 75%         |
| Financial Type & Date Filter         | 10 reads        | 2 reads        | 80%         |

Table 2 — Logical read comparison before and after indexing

### 2.2 Execution Time Impact — Latency

| Scenario              | Baseline Latency | Optimised Latency | Reduction |
|-----------------------|------------------|-------------------|-----------|
| Dashboard Page Load   | 450 ms           | 120 ms            | 73%       |
| Audit Log Search      | 210 ms           | 45 ms             | 79%       |
| Financial Aggregation | 180 ms           | 35 ms             | 81%       |

Table 3 — End-to-end latency comparison

### 3. Write Overhead Analysis

| Table               | Index Count | Write Overhead | Assessment                            |
|---------------------|-------------|----------------|---------------------------------------|
| EmergencyReports    | 5 indexes   | ~23% on INSERT | Acceptable — moderate write frequency |
| AuditLog            | 2 indexes   | ~9% on INSERT  | Minimal — intentionally light         |
| ResourceAllocations | 1 index     | ~5% on INSERT  | Negligible                            |
| FinanceTransactions | 1 index     | ~5% on INSERT  | Negligible                            |

Table 4 — Write overhead analysis per table

## 4. Conclusion

---

The indexing strategy delivers approximately **80% reduction in database I/O** for critical query paths while maintaining write overhead below **25%** — within acceptable limits for a disaster management system.

- **Composite indexes** outperform single-column indexes for multi-condition WHERE clauses common in dashboard filters.
- High-frequency write tables (AuditLog) are **intentionally lightly indexed** to preserve insert performance.
- The strategy keeps the system responsive as the disaster event database grows over time.